
The Complete Developer Guide

BY JANE SMITH

2024-01-15

Table of Contents

Introduction

- About This Book
- How to Read This Guide
- Getting Started

Mathematics

- Inline Math
- Display Math
- Summation and Products
- Matrix Notation

Diagrams

- Flowchart
- Sequence Diagram
- Class Diagram
- Entity Relationship
- State Diagram
- Multi-line Labels and Quotes

Code Examples

- JavaScript
- Python
- SQL
- Bash
- Feature Comparison Table

Appendix A: Configuration Reference

Appendix B: Glossary

Introduction

Welcome to **The Complete Developer Guide**. This book covers mathematics, diagrams, and code examples rendered with professional typesetting.

About This Book

This guide demonstrates the full feature set of md2pdf-book, including:

- LaTeX math rendering via KaTeX
- Mermaid diagram support (flowcharts, sequences, class diagrams, ERDs, state machines)
- Syntax-highlighted code blocks for JavaScript, Python, SQL, and more
- Automated table of contents with dot leaders
- Professional cover page with gradient background

How to Read This Guide

Each chapter is self-contained and focuses on one feature area:

Chapter	Topic
Introduction	Overview and structure
Mathematics	LaTeX inline and display math
Diagrams	Mermaid diagram types
Code	Syntax highlighting and tables

NOTE

All code examples in this book are runnable and have been tested against the referenced library versions.

Getting Started

To compile this book yourself, run:

BASH

```
md2pdf-book book.yaml
```

Or generate your own starter config with:

BASH

```
md2pdf-book --init
```

TIP

Callout boxes like this one are written with GitHub-style syntax — `> [!NOTE]`, `> [!TIP]`, `> [!IMPORTANT]`, `> [!WARNING]`, or `> [!CAUTION]` as the first line of a blockquote.

WARNING

Unrecognized markers, or a plain `>` quote with no marker at all, still render as an ordinary blockquote — nothing breaks if you mix both styles.

Footnotes are also supported using standard Pandoc-style syntax¹, including inside admonitions and tables.

1. Write `text[^id]` for the reference and `[^id]: definition` anywhere in the file — they're collected and numbered automatically at the bottom of the page.

Mathematics

This chapter demonstrates LaTeX math rendering via KaTeX.

Inline Math

The famous identity $e^{i\pi} + 1 = 0$ connects the five most important constants in mathematics.

Einstein's mass-energy equivalence is expressed as $E = mc^2$, where m is mass and c is the speed of light.

For a normal distribution with mean μ and standard deviation σ , the probability density function is $f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$.

Display Math

The quadratic formula gives the roots of $ax^2 + bx + c = 0$:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The Fourier transform of a function $f(t)$:

$$\hat{f}(\xi) = \int_{-\infty}^{\infty} f(t) e^{-2\pi i t \xi} dt$$

Maxwell's equations in differential form:

$$\nabla \cdot \mathbf{E} = \frac{\rho}{\epsilon_0}$$

$$\nabla \times \mathbf{B} = \mu_0 \mathbf{J} + \mu_0 \epsilon_0 \frac{\partial \mathbf{E}}{\partial t}$$

Summation and Products

The sum of the first n natural numbers:

$$\sum_{k=1}^n k = \frac{n(n+1)}{2}$$

The definition of e as a limit:

$$e = \lim_{n \rightarrow \infty} \left(1 + \frac{1}{n}\right)^n = \sum_{n=0}^{\infty} \frac{1}{n!}$$

Matrix Notation

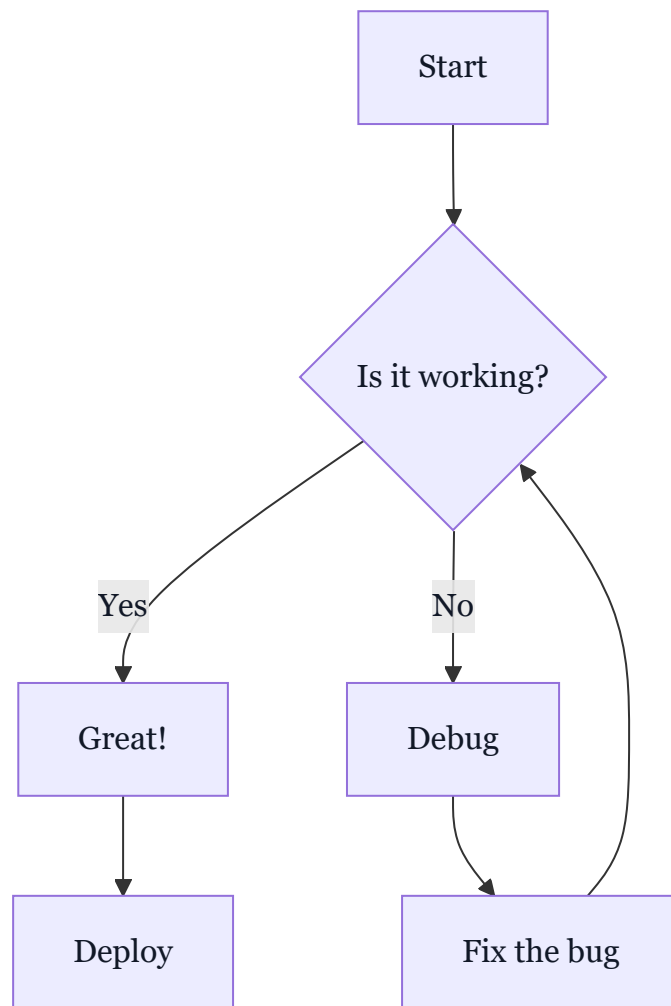
A 2×2 matrix determinant:

$$\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc$$

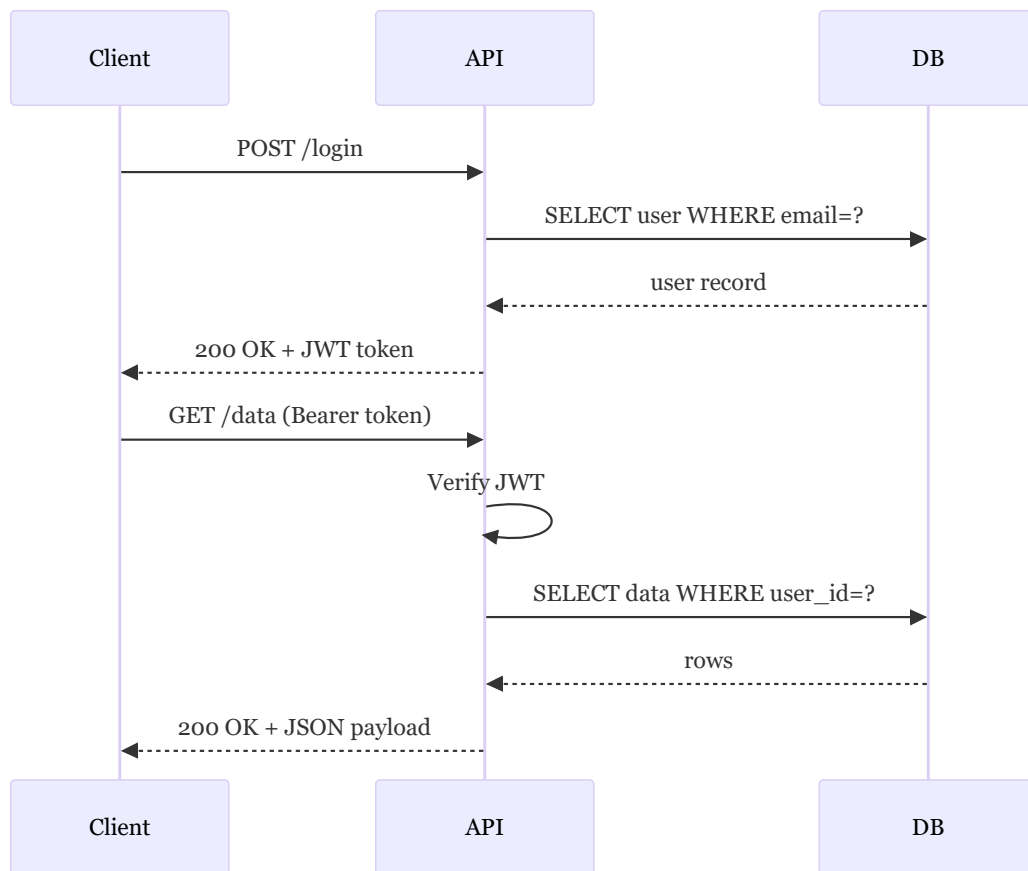
Diagrams

This chapter demonstrates Mermaid diagram rendering across multiple diagram types.

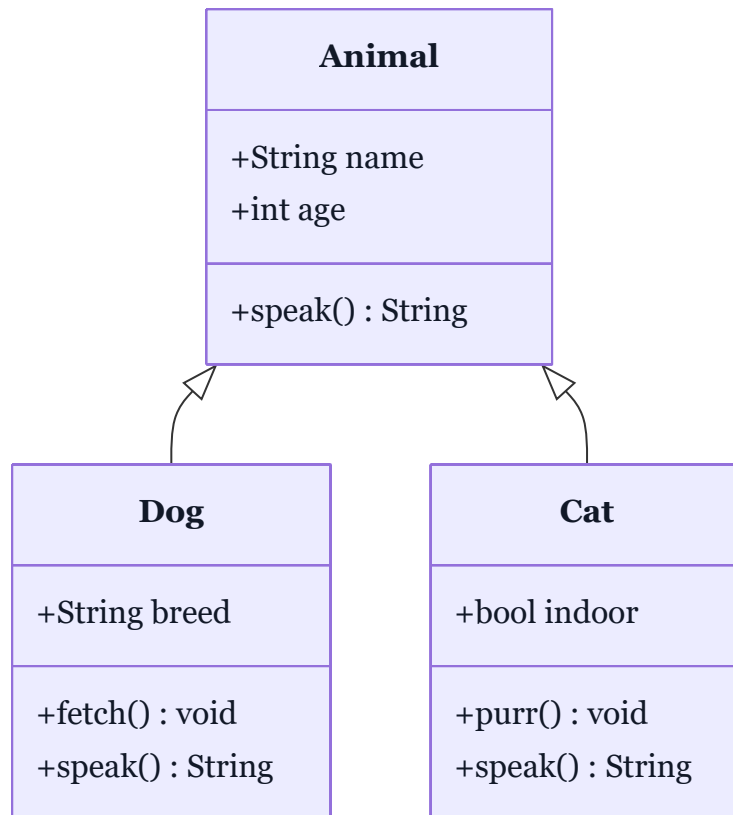
Flowchart



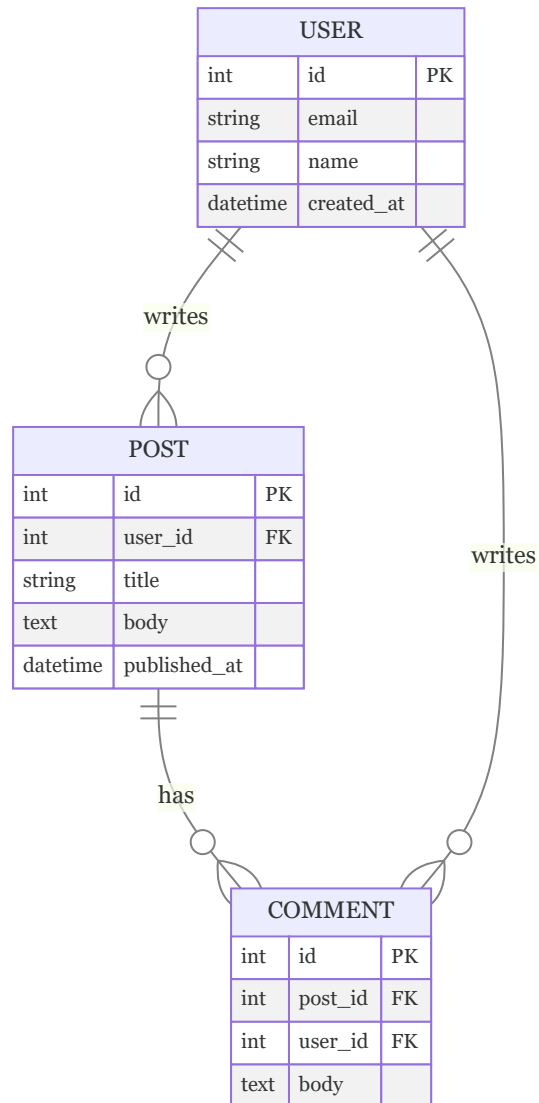
Sequence Diagram



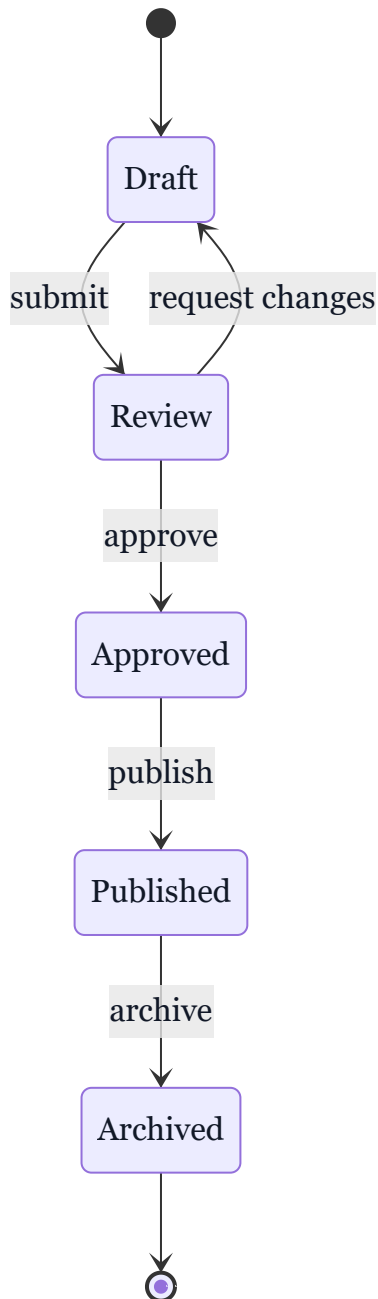
Class Diagram



Entity Relationship

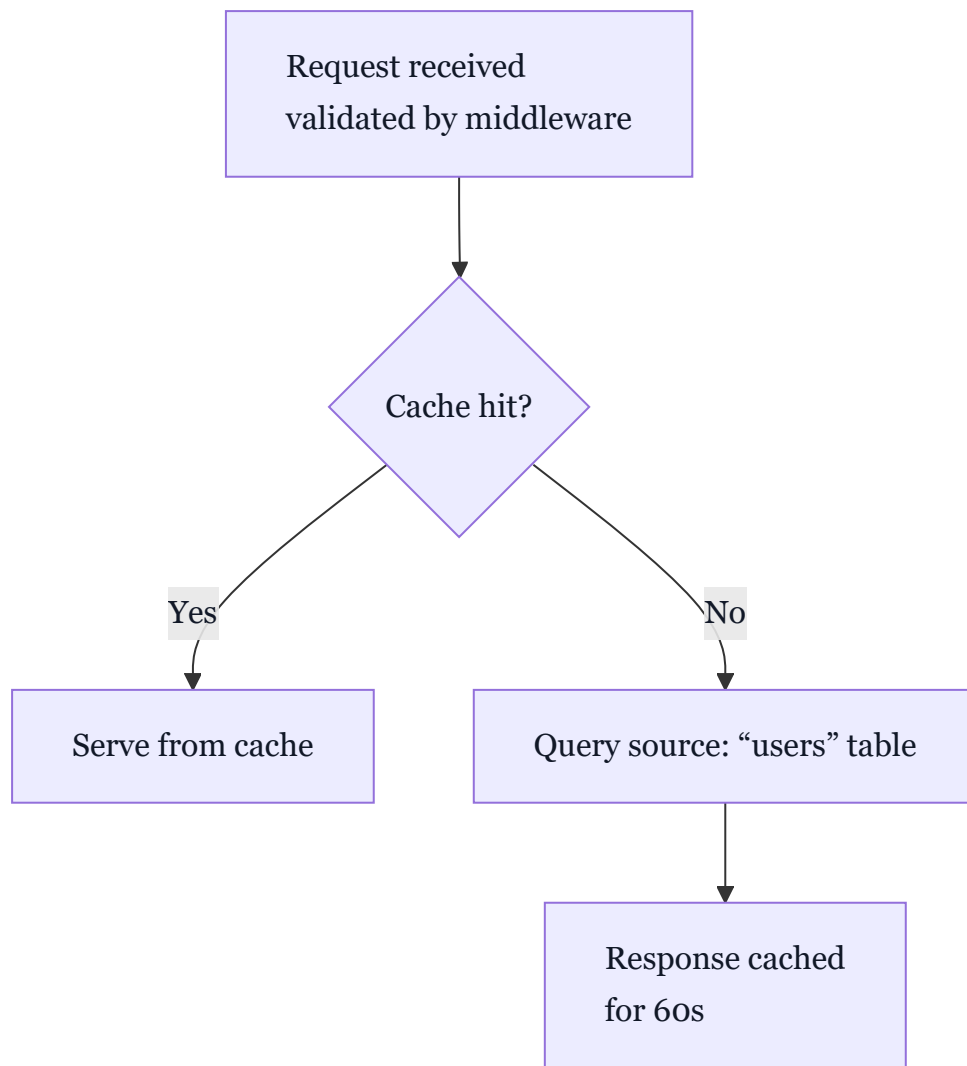


State Diagram



Multi-line Labels and Quotes

Node labels can use `\n` for line breaks and escaped quotes for embedded quotation marks — both are rewritten automatically so Mermaid renders them correctly instead of showing literal backslashes:



Code Examples

This chapter demonstrates syntax-highlighted code blocks across multiple languages.

JavaScript

Async/await with structured error handling:

JAVASCRIPT

```
// Fetch a user from the API with full error handling
async function fetchUser(id) {
  try {
    const response = await fetch(`/api/users/${id}`);

    if (!response.ok) {
      throw new Error(`HTTP ${response.status}: ${response.statusText}`);
    }

    const user = await response.json();
    return user;
  } catch (err) {
    console.error('Failed to fetch user:', err.message);
    throw err;
  }
}

// Parallel fetching with Promise.all
async function fetchDashboard(userId) {
  const [user, posts, notifications] = await Promise.all([
    fetchUser(userId),
    fetch(`/api/users/${userId}/posts`).then(r => r.json()),
    fetch(`/api/notifications`).then(r => r.json()),
  ]);

  return { user, posts, notifications };
}
```

Python

A typed dataclass with password hashing:

PYTHON

```
from dataclasses import dataclass
from typing import Optional
import hashlib
import secrets

@dataclass
class User:
    id: int
    email: str
    name: str
    password_hash: Optional[str] = None

    @classmethod
    def create(cls, id: int, email: str, name: str, password: str) ->
    "User":
        salt = secrets.token_hex(16)
        hashed = hashlib.sha256(f"{salt}{password}".encode()).hexdigest()
        return cls(id=id, email=email, name=name, password_hash=f"{salt}:
{hashed}")

    def verify_password(self, password: str) -> bool:
        if not self.password_hash:
            return False
        salt, expected = self.password_hash.split(":", 1)
        actual = hashlib.sha256(f"{salt}{password}".encode()).hexdigest()
        return secrets.compare_digest(actual, expected)
```

SQL

Schema creation with indexed queries:

SQL

```
-- Users table with audit timestamps
CREATE TABLE users (
  id          SERIAL PRIMARY KEY,
  email       VARCHAR(255) NOT NULL UNIQUE,
  name        VARCHAR(100) NOT NULL,
  created_at  TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at  TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE INDEX idx_users_email ON users (email);

-- Get top authors by published post count
SELECT
  u.id,
  u.name,
  u.email,
  COUNT(p.id)      AS post_count,
  MAX(p.published_at) AS last_published
FROM users u
LEFT JOIN posts p
  ON p.user_id = u.id
  AND p.published_at IS NOT NULL
GROUP BY u.id, u.name, u.email
ORDER BY post_count DESC
LIMIT 20;
```

Bash

A deployment script with safety checks:

BASH

```
#!/usr/bin/env bash
set -euo pipefail

APP="my-app"
TARGET="/var/www/${APP}"
BACKUP="/var/backups/${APP}-${date +%Y%m%d-%H%M%S}"

echo "→ Backing up current release to ${BACKUP}"
cp -r "${TARGET}" "${BACKUP}"

echo "→ Pulling latest changes"
git -C "${TARGET}" pull --ff-only origin main

echo "→ Installing dependencies"
npm ci --prefix "${TARGET}" --omit=dev

echo "→ Restarting service"
systemctl restart "${APP}"

echo "✅ Deployment complete"
```

Feature Comparison Table

Feature	KaTeX	MathJax	MathML
Render speed	Fast	Slow	Native
Browser support	All	All	Partial
TeX coverage	Most	Full	Limited
PDF output	✅	✅	⚠️
Bundle size	Small	Large	None
Sync rendering	✅	❌	N/A

Appendix A: Configuration Reference

Appendices are a common case for wanting several chapters in a single file instead of one file per chapter. Each top-level (#) heading below starts on its own page automatically, exactly like a chapter defined in its own file.

Option	Default	Description
page_size	A4	A4, A3, Letter, or Legal
orientation	portrait	portrait or landscape
chapter_breaks	true	Page break before each H1 heading

Need a break in the middle of a chapter without starting a new one? Drop an `<!-- pagebreak -->` comment (or a lone `\pagebreak` line) on its own line, as was done just above this paragraph.

Appendix B: Glossary

Chapter : A section of the book starting at a top-level (#) heading. Chapters no longer need to live in separate files — a single markdown file can define as many as it likes.

Manual page break : An `<!-- pagebreak -->` marker used to force a new page inside a chapter.